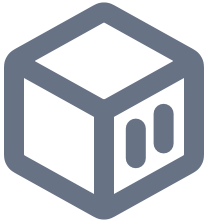


[Explore](#) / [gvenzl](#) / oracle-free

gvenzl/oracle-free

By [gvenzl](#) • Updated 2 months ago

Oracle Database Free for everyone! :)

IMAGE

DATABASES & STORAGE

☆93 ↓ 5M+

Overview

Tags

oci-oracle-free

Oracle Database Free Container images.

Table of Contents

- [Supported tags](#)
- [Quick Start](#)
- [Users of these images](#)
- [How to use these images](#)
 - [Image flavors](#)

Tag summary

Recent tags

full-faststart



Content type

Image

Digest

sha256:08d9bec58...

Size

- [Environment variables](#)
- [GitHub Actions](#)
- [Docker Compose](#)
- [Database users](#)
- [Pluggable databases](#)
- [Container secrets](#)
- [Initialization scripts](#)
- [Startup scripts](#)
- [Configuration scripts](#)

- [Feedback](#)

Supported tags

Tag	Status
latest[-faststart]	Always Supported
slim[-faststart]	Always Supported
full[-faststart]	Always Supported
23[-faststart] 23[-slim][-faststart] 23[-full][-faststart]	Supported
23.26.1[-faststart] 23.26.1[-slim][-faststart] 23.26.1[-full][-faststart]	Supported

2.6 GB

Last updated

2 months ago

```
docker pull gvenzl/oracle
-free:full-faststart
```



Tag	Status
23.26.0[-faststart] 23.26.0[-slim][-faststart] 23.26.0[-full][-faststart]	● Deprecated
23.9[-faststart] 23.9[-slim][-faststart] 23.9[-full][-faststart]	● Unsupported
23.8[-faststart] 23.8[-slim][-faststart] 23.8[-full][-faststart]	● Unsupported
23.7[-faststart] 23.7[-slim][-faststart] 23.7[-full][-faststart]	● Unsupported
23.6[-faststart] 23.6[-slim][-faststart] 23.6[-full][-faststart]	● Unsupported
23.5[-faststart] 23.5[-slim][-faststart] 23.5[-full][-faststart]	● Unsupported
23.4[-faststart] 23.4[-slim][-faststart] 23.4[-full][-faststart]	● Unsupported

Tag	Status
23.3[-faststart] 23.3[-slim][-faststart] 23.3[-full][-faststart]	● Unsupported
23.2[-faststart] 23.2[-slim][-faststart] 23.2[-full][-faststart]	● Unsupported

Tags

Tags in `[]` denote tag options, for example, `23[-slim][-faststart]` means there are the following tags:

- `23`
- `23-slim`
- `23-faststart`
- `23-slim-faststart`

Support status

Status	Meaning
● Always Supported	These images will always be present and receive bug fixes and regular updates.
● Supported	These images are currently supported and receive fixes and regular updates.

Status	Meaning
 Deprecated	These images are deprecated and will only receive bug fixes. Upgrading to a newer images is strongly advised.
 Unsupported	These images are unsupported, will receive no further updates and may be removed at any time. Using these images is strongly discouraged!

Quick Start

Run a new database container (data is removed when the container is removed, but kept throughout container restarts):

```
docker run -d -p 1521:1521 -e ORACLE_PASSWORD=<your password> gvenzl/oracle-free
```

Run a new persistent database container (data is kept throughout container lifecycles):

```
docker run -d -p 1521:1521 -e ORACLE_PASSWORD=<your password> -v oracle-volume:/opt/oracle/oradata gvenzl/oracle-free
```

Reset database `SYS` and `SYSTEM` passwords:

```
docker exec <container name|id> resetPassword <your password>
```

Oracle Database Free on Apple MacBooks with ARM M-chips

Starting with Oracle Database 23.5 Free, Oracle provides ARM ports for Oracle Database Free. Multi-platform (multi-arch) images are provided starting with 23.5.

Users of these images

We are proud of the following users of these images:

- [Benthos](#)  [`c29f81d` ]
- [Hibernate Reactive](#)  [`0af4ebc`] 
- [Ibis](#)  [`b568a81` ]
- [JobRunr](#)  [`675061e` ]
- [jOOQ](#)  [`Twitter` ]
- [Quarkus](#)  [`546922c` ]
- [Ruby on Rails ActiveRecord adapter](#)  [`deb214d` ]
- [Spring Data](#)  [`3cac9d1` ]
- [Micronaut](#)  [`37882de` ]
- [utPLSQL](#)  [`0497dcf` ]

If you are using these images and would like to be listed as well, please open an [issue on GitHub](#) .

How to use these images

Image flavors

Flavor	Extension	Description	Use cases
Slim	<code>-slim</code>	An image focussed on smallest possible image size instead of additional functionality.	Wherever small image sizes are important but advanced functionality of Oracle Database is not needed.

Flavor	Extension	Description	Use cases
Regular	[None]	A well-balanced image between image size and functionality. Recommended for most use cases.	Recommended for most use cases.
Full	<code>-full</code>	An image containing all functionality as provided by the Oracle Database installation.	Best for extensions and/or customizations.
Faststart	<code>*-faststart</code>	The same image flavor as above but with an already expanded and ready-to-go database inside the image. This image trades image size on disk for a faster database startup time.	Best for (automated) test scenarios where the image is pulled once and many containers are started and torn down with no need for persistence (container volumes).

See [ImageDetails.md](#) [↗](#) for a full list of changes that have been made to the Oracle Database and OS installation in each individual image flavor.

Environment variables

Environment variables allow you to customize your container. These variables will only be considered during the database initialization (first container startup).

`ORACLE_PASSWORD`

This variable is mandatory for the first container startup and specifies the password for the Oracle Database `SYS` and `SYSTEM` users.

ORACLE_RANDOM_PASSWORD

Optional. Set this variable to a non-empty value, like `yes`, to generate a random initial password for the `SYS` and `SYSTEM` users. The generated password will be printed to stdout (`ORACLE_PASSWORD FOR SYS AND SYSTEM: ...`).

ORACLE_DATABASE

Optional. Set this variable to a non-empty string to either create a new pluggable database or plug in an existing PDB found in `pdb-plug/<name>.pdb` with the name specified in this variable. Multiple pluggable databases are created when providing a comma-separated list, for example, `ORACLE_DATABASE=PDB1,PDB2,PDB3`.

Note: creating a new pluggable database will add to the initial container startup time. If you do not want that additional startup time, use the already existing `FREEPDB1` database instead.

APP_USER

Optional. Set this variable to a non-empty string to create a new database schema user with the name specified in this variable. For 18c and onwards, the user will be created in the default `FREEPDB1` pluggable database. If `ORACLE_DATABASE` has been specified, the user will also be created in that pluggable database. This variable requires `APP_USER_PASSWORD` or `APP_USER_PASSWORD_FILE` to be specified as well.

APP_USER_PASSWORD

Optional. Set this variable to a non-empty string to define a password for the database schema user specified by `APP_USER`. This variable requires `APP_USER` to be specified as well.

GitHub Actions

Action

These images can be used via the GitHub [Setup Oracle DB Free](#) action available on the GitHub Actions Marketplace. The basic usage can be:

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: gvenzl/setup-oracle-free@v1
      with:
        app-user: <username>
        app-user-password: <password>
```

Service Container

Alternatively, the images can be used directly as a [Service Container](#) within a GitHub Actions workflow. Below is an example service definition for your GitHub Actions YAML file:

```
services:

  # Oracle service (label used to access the service container)
  oracle:

    # Docker Hub image (feel free to change the tag "latest" to
    any other available one)
    image: gvenzl/oracle-free:latest

    # Provide passwords and other environment variables to conta
    iner
    env:
      ORACLE_RANDOM_PASSWORD: true
      APP_USER: my_user
      APP_USER_PASSWORD: password_i_should_change

    # Forward Oracle port
    ports:
      - 1521:1521
```

```
# Provide healthcheck script options for startup
options: >-
  --health-cmd healthcheck.sh
  --health-interval 10s
  --health-timeout 5s
  --health-retries 10
```

After your service is created, you can connect to it via the following properties:

- Hostname:
 - `oracle` (from within another container)
 - `localhost` or `127.0.0.1` (from the host directly)
- Port: `1521`
- Service name: `FREEPDB1`
- Database App User: `my_user`
- Database App Password: `password_i_should_change`

If you amend the variables above, here is some more useful info:

- Ports: you can access the port dynamically via `${{ job.services.oracle.ports[1521] }}`. This is helpful when you do not want to specify a given port via `- 1521/tcp` instead of `- 1521:1521`. Note that the `oracle` refers to the service name in the yaml file. If you call your service differently, you will also have to change `oracle` here to that other service name.
- Database Admin User: `system`
- Database Admin User Password: `$ORACLE_PASSWORD`
- Database App User: `$APP_USER`
- Database App User Password: `$APP_USER_PASSWORD`
- Example JDBC connect string with dynamic port allocation:
`jdbc:oracle:thin:@localhost:${{ job.services.oracle.ports[1521] }}/FREEPDB1`

Docker Compose

The images can be used in a [Docker Compose](#)  setup to provide a local development database or facilitate automated testing. Below is an example service definition for your Docker Compose YAML file:

```
version: "3.8"
services:
  # Name of the Docker Compose service
  oracle:
    # Docker Hub image (feel free to change the tag "latest" to any other available one)
    image: gvenzl/oracle-free:latest
    # Forward Oracle port to localhost
    ports:
      - "1521:1521"
    # Provide passwords and other environment variables to the container
    environment:
      ORACLE_PASSWORD: sys_user_password
      APP_USER: my_user
      APP_USER_PASSWORD: password_i_should_change
    # Customize healthcheck script options for startup
    healthcheck:
      test: ["CMD", "healthcheck.sh"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 5s
      start_interval: 5s
    # Mount a local SQL file to initialize your schema at startup
    volumes:
      - my-init.sql:/container-entrypoint-initdb.d/my-init.sql:ro
```

After your container is up and running, you can connect to it via the following properties:

- Hostname:
 - `oracle` (from within another service defined in the compose file)
 - `localhost` or `127.0.0.1` (from the host directly)

- Port: 1521
- Service name: FREEPDB1
- Database App User: my_user
- Database App Password: password_i_should_change

To know more about initialization scripts, please refer to the [Initialization scripts section](#).

Database users

The image provides a built-in command `createAppUser` to create additional Oracle Database users with standard privileges. The same command is also executed when the `APP_USER` environment variable is specified. If you need just one additional database user for your application, the `APP_USER` environment variable is the best approach. However, if you need multiple users, you can execute the command for each individual user directly:

Usage :

```
createAppUser APP_USER APP_USER_PASSWORD [TARGET_PDB]
```

APP_USER: the user name of the new user

APP_USER_PASSWORD: the password for that user

TARGET_PDB: the target pluggable database the user should be created in, default FREEPDB1

Example:

```
docker exec <container name|id> createAppUser <your app user> <your app user password> [<your target PDB>]
```

The command can also be invoked inside initialization and/or startup scripts.

Pluggable databases

Automatically plug-in one or more PDBs by providing the `<PDB_NAME>.pdb` file(s) in the `/pdb-plug` folder inside the container and list the PDB name(s) in `ORACLE_DATABASE`.

Container secrets

As an alternative to passing sensitive information via environment variables, `_FILE` may be appended to some of the previously listed environment variables, causing the initialization script to load the values for those variables from files present in the container. In particular, this can be used to load passwords from Container/Docker secrets stored in `/run/secrets/<secret_name>` files. For example:

```
docker run -d --name some-oracle -e ORACLE_PASSWORD_FILE=/run/secrets/oracle-passwd gvenz1/oracle-free
```

This mechanism is supported for:

- `APP_USER_PASSWORD`
- `ORACLE_PASSWORD`
- `ORACLE_DATABASE`

Note: there is a significant difference in how containerization technologies handle secrets. For more information on that topic, please consult the official containerization technology documentation:

- [Docker](#) 
- [Podman](#) 
- [Kubernetes](#) 

Initialization scripts

If you would like to perform additional initialization of the database running in a container, you can add one or more `*.sql`, `*.sql.gz`, `*.sql.zip` or `*.sh` files under `/container-entrypoint-initdb.d` (creating the directory if necessary). After the database setup is completed, these files will be executed automatically in alphabetical order.

The directory can include sub-directories which will be traversed recursively in alphabetical order alongside the files. The container does not give any priority to files or directories, meaning that whatever comes next in alphabetical order will be processed next. If it is a file

it will be executed, if it is a directory it will be traversed. To guarantee the order of execution, consider using a clear prefix in your file and directory names like numbers `001_` , `002_` . This will also make it easier for any user to understand which script is supposed to be executed in what order.

The `*.sql` , `*.sql.gz` and `*.sql.zip` files *will be executed in SQL*Plus as the SYS user connected to the Oracle instance (FREE)*. This allows users to modify instance parameters, create new pluggable databases, tablespaces, users and more as part of their initialization scripts. *If you want to initialize your application schema, you first have to connect to that schema inside your initialization script!* Compressed files will be uncompressed on the fly, allowing for e.g. bigger data loading scripts to save space.

Executable `*.sh` files will be run in a new shell process while non-executable `*.sh` files (files that do not have the Linux `ex` executable permission set) will be sourced into the current shell process. The main difference between these methods is that sourced shell scripts can influence the environment of the current process and should generally be avoided. However, sourcing scripts allows for execution of these scripts even if the executable flag is not set for the files containing them. This basically avoids the "why did my script not get executed" confusion.

Note: scripts in `/container-entrypoint-initdb.d` are only run the first time the database is initialized; any pre-existing database will be left untouched on container startup.

Note: you can also put files under the `/docker-entrypoint-initdb.d` directory. This is kept for backward compatibility with other widely used container images but should generally be avoided. Do not put files under `/container-entrypoint-initdb.d` and `/docker-entrypoint-initdb.d` as this would cause the same files to be executed twice!

Warning: if a command within the sourced `/container-entrypoint-initdb.d` scripts fails, it will cause the main entrypoint script to exit and stop the container. It also may leave the database in an incomplete initialized state. Make sure that shell scripts handle error situations gracefully and ideally do not source them!

Warning: do not exit executable `/container-entrypoint-initdb.d` scripts with a non-zero value (using e.g. `exit 1;`) unless it is desired for a container to be stopped! A non-zero return value will tell the main entrypoint script that something has gone wrong and that the container should be stopped.

Example

The following example installs the [countries, cities and currencies sample data set](#) under a new user `TEST` into the database:

```
[gvenzl@localhost init_scripts]$ pwd
/home/gvenzl/init_scripts

[gvenzl@localhost init_scripts]$ ls -al
total 12
drwxrwxr-x  2 gvenzl gvenzl  61 Apr  3 20:11 .
drwx----- 19 gvenzl gvenzl 4096 Apr  3 20:11 ..
-rw-rw-r--  1 gvenzl gvenzl  134 Apr  3 20:10 1_create_user.sql
-rwxrwxr-x  1 gvenzl gvenzl  164 Apr  3 20:11 2_create_data_model.sh

[gvenzl@localhost init_scripts]$ cat 1_create_user.sql
ALTER SESSION SET CONTAINER=FREEPDB1;

CREATE USER TEST IDENTIFIED BY test QUOTA UNLIMITED ON USERS;

GRANT CONNECT, RESOURCE TO TEST;

[gvenzl@localhost init_scripts]$ cat 2_create_data_model.sh
curl -LJO https://raw.githubusercontent.com/gvenzl/sample-data/master/countries-cities-currencies/install.sql

sqlplus -s test/test@//localhost/FREEPDB1 @install.sql

rm install.sql
```

As the execution happens in alphabetical order, numbering the files will guarantee the execution order. A new container started up with `/home/gvenzl/init_scripts` pointing to `/container-entrypoint-initdb.d` will then execute the files above:

```

docker run --name test \
> -p 1521:1521 \
> -e ORACLE_RANDOM_PASSWORD="y" \
> -v /home/gvenzl/init_scripts:/container-entrypoint-initdb.d \
gvenzl/oracle-free:23-slim
CONTAINER: starting up...
CONTAINER: first database startup, initializing...
...
CONTAINER: Executing user-defined scripts...
CONTAINER: running /container-entrypoint-initdb.d/1_create_user.sql
...

Session altered.

User created.

Grant succeeded.

CONTAINER: DONE: running /container-entrypoint-initdb.d/1_create_user.sql

CONTAINER: running /container-entrypoint-initdb.d/2_create_data_model.sh ...
% Total    % Received % Xferd  Average Speed   Time    Time     Ti
me  Current
                        Dload  Upload   Total   Spent    Le
ft  Speed
100 115k  100  115k    0     0  460k      0 --:--:-- --:--:-- --:--
-:-- 460k

Table created.
...
Table                provided actual
-----
regions                7         7

```


countries	196	196
cities	204	204
currencies	146	146
currencies_countries	203	203

Thank you!

The installation is finished, please check the verification output above!

If the 'provided' and 'actual' row counts match, the installation was successful

.

If the row counts do not match, please check the above output for error messages

.

CONTAINER: DONE: running /container-entrypoint-initdb.d/2_create_data_model.sh

CONTAINER: DONE: Executing user-defined scripts.

#####

DATABASE IS READY TO USE!

#####

...

As a result, one can then connect to the new schema directly:

```
[gvenzl@localhost init_scripts]$ sqlplus test/test@//localhost/FREEDB1
```

```
SQL> select * from countries where name = 'Austria';
```

COUNTRY_ID	COUNTRY_CODE	NAME	OFFICIAL_NAME	POPULATION	AREA_
SQ_KM	LATITUDE	LONGITUDE	TIMEZONE	REGION_ID	
AUT	AT	Austria	Republic of Austria	8793000	
83871	47.33333	13.33333	Europe/Vienna	EU	

SQL>

Startup scripts

If you would like to perform additional action after the database running in a container has been started, you can add one or more `*.sql`, `*.sql.gz`, `*.sql.zip` or `*.sh` files under `/container-entrypoint-startdb.d` (creating the directory if necessary). After the database is up and ready for requests, these files will be executed automatically in alphabetical order.

The execution order and implications are the same as with the [Initialization scripts](#) described above.

Note: you can also put files under the `/docker-entrypoint-startdb.d` directory. This is kept for backward compatibility with other widely used container images but should generally be avoided. Do not put files under `/container-entrypoint-startdb.d` and `/docker-entrypoint-startdb.d` as this would cause the same files to be executed twice!

Note: if the database inside the container is initialized (started for the first time), startup scripts are executed after the setup scripts.

Warning: files placed in `/container-entrypoint-startdb.d` are always executed after the database in a container is started, including pre-created databases. Use this mechanism only if you wish to perform a certain task always after the database has been (re)started by the container.

Configuration scripts

If you would like to change the configuration of the database, several configuration scripts can be found in [config-scripts](#).

Feedback

If you have questions or constructive feedback about these images, please submit a ticket at github.com/gvenzl/oci-oracle-free [↗](#).



Why

[Overview](#)
[What is a Container](#)

Products

[Product Overview](#)

Product Offerings

[Docker Desktop](#)
[Docker Hub](#)

Features

[Container Runtime](#)
[Developer Tools](#)
[Docker App](#)
[Kubernetes](#)

Developers

[Getting Started](#)
[Play with Docker](#)
[Community](#)
[Open Source](#)
[Documentation](#)

Company

[About Us](#)
[Resources](#)
[Blog](#)
[Customers](#)
[Partners](#)
[Newsroom](#)
[Events and Webinars](#)
[Careers](#)
[Contact Us](#)
[System Status](#) [↗](#)

© 2026 Docker, Inc. All rights reserved. | [Terms of Service](#) | [Subscription Service Agreement](#) | [Privacy](#)

| [Legal](#)

[Cookies Settings](#)

